

Hiding Tensor-Parallel Attention Reduce Communication Latency with Tile Pipelining

Sucheer Maddury Aden Zhao

CS 5220: Applications of Parallel Computers
Spring 2026

1 Hypothesis

Tensor parallelism is the standard to distribute large transformer layers across multiple GPUs, but it introduces collective communication on the critical path of each layer. In a tensor-parallel attention block, each rank computes a subset of attention heads, applies a row-sharded output projection, and then participates in an all-reduce to form the final output. In a conventional implementation, the attention computation, output projection, and all-reduce occur in sequence, so the collective latency is exposed after local computation completes.

This project explores whether FlashAttention-style query tiling can expose exact, communication-ready output tiles early enough to overlap tensor-parallel all-reduce with later attention computation. The central question is:

Can a tensor-parallel attention reduce end-to-end latency by finalizing partial output tiles and hide rank-to-rank communication latency by overlapping reduction with the computation of the next tile?

The idealized performance model is

$$t_{\text{serialized}} \approx t_{\text{compute}} + t_{\text{allreduce}}, \quad (1)$$

$$t_{\text{overlap}} \approx \max(t_{\text{compute}}, t_{\text{allreduce}}). \quad (2)$$

In practice, this model only holds if the cost hidden by overlap is larger than the extra overhead from tiling, additional kernel launches, CUDA event synchronization, output projection calls, and nonblocking MPI collectives. Our empirical study compares three execution modes: a serialized baseline, a tiled but non-overlapped baseline, and a tiled overlapped implementation.

2 Context and Contributions

This project was completed as a CS 5220 final project. It is not a sub-project within a larger research effort, and it is not being reused for another course. The implementation is available at:

https://github.com/sumaddury/Fused_Attention_Allreduce

Sucheer focused on the FlashAttention side of the project: implementing the tiled attention kernel, reasoning about the online softmax update, validating kernel correctness, and tuning legal

tile shapes such as B_M and B_N . Aden focused on the runtime and systems side: implementing the serialized and tiled baselines, building the C++/CUDA/MPI communication pipeline, adding nonblocking all-reduce support, writing the profiling and experiment scripts, and organizing the report analysis.

3 Prior Work

3.1 Tensor Parallelism

Large transformer models often exceed the memory and compute capacity of a single GPU. Tensor parallelism splits weights and intermediate activations across ranks; Megatron-LM popularized tensor, pipeline, and data parallelism as composable strategies for training large models on GPU clusters [1, 2]. In tensor-parallel transformer blocks, ranks compute partial matrix products or subsets of attention heads and then synchronize with collective communication.

We focus on the attention output projection, which creates the all-reduce our project tries to hide. Let $X \in \mathbb{R}^{S \times d}$ be the input sequence, with H heads, R ranks, and per-head dimension $d_h = d/H$. Each head computes

$$A_h = \text{softmax} \left(\frac{Q_h K_h^T}{\sqrt{d_h}} \right) V_h, \quad A_h \in \mathbb{R}^{S \times d_h}. \quad (3)$$

Rank r concatenates H/R local heads into $A^{(r)} \in \mathbb{R}^{S \times d/R}$. The output projection is row-sharded, so rank r owns $W_O^{(r)} \in \mathbb{R}^{d/R \times d}$ and computes

$$P^{(r)} = A^{(r)} W_O^{(r)}, \quad P^{(r)} \in \mathbb{R}^{S \times d}. \quad (4)$$

The final output is the sum over ranks:

$$O = \sum_{r=0}^{R-1} P^{(r)}. \quad (5)$$

This final reduction motivates our project: if $P^{(r)}$ is produced only after all local attention work completes, then the all-reduce communication latency is fully exposed.

3.2 FlashAttention

FlashAttention is an exact online attention algorithm that avoids materializing the full score matrix by streaming key/value tiles through on-chip memory [3]. For query tile i and key/value tile j , define $S_{ij} = Q_i K_j^T / \sqrt{d_h}$. The kernel maintains the running row maximum $m_i^{(j)}$, softmax denominator $\ell_i^{(j)}$, and unnormalized output accumulator $\tilde{O}_i^{(j)}$:

$$m_i^{(j)} = \max \left(m_i^{(j-1)}, \text{rowmax}(S_{ij}) \right). \quad (6)$$

$$\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum} \left(e^{S_{ij} - m_i^{(j)}} \right), \quad (7)$$

$$\tilde{O}_i^{(j)} = \text{diag} \left(e^{m_i^{(j-1)} - m_i^{(j)}} \right) \tilde{O}_i^{(j-1)} + e^{S_{ij} - m_i^{(j)}} V_j. \quad (8)$$

After all $T_N = S/B_N$ key/value tiles have been consumed, the query tile output is exact:

$$O_i = \text{diag} \left(\ell_i^{(T_N)} \right)^{-1} \tilde{O}_i^{(T_N)}. \quad (9)$$

Our pipeline doesn’t depend on FlashAttention for correctness, as we need to sweep over all the T_N key-value tiles in a single query tile block before we can apply the output projection and start communication. However, FlashAttention is instrumental so that we can fit key/value blocks inside shared memory, which is significantly faster than lower memory levels. With FlashAttention, we can compute partial results over the key/value dimension while easing memory boundedness.

3.3 Computation-Communication Overlap

Our design is related to prior systems that overlap transformer computation and communication. ISO proposes intra-sequence overlap for LLM inference by reordering sequence-dependent and communication-heavy work so communication can proceed concurrently with useful computation [5]. Our project applies the same principle inside a FlashAttention-style attention layer: query tiles, rather than whole requests or layer chunks, become the unit of overlap.

4 Implementation Methodology

4.1 Execution Modes

We implemented three execution modes to measure the cost/benefit of tiling and the benefit of overlap. All execution modes used the same underlying FlashAttention-based kernel, whose strategy will be described shortly.

Serialized baseline. The serialized mode computes all local attention output, applies the output projection to the full local output, and then performs one blocking all-reduce.

Tiled non-overlapped baseline. The tiled mode computes and projects one query tile at a time, but performs a synchronous/blocking all-reduction instead of async. It isolates the overhead introduced by per-tile reduction launches and multiple projection calls.

Tiled overlapped implementation. The overlapped mode computes a query tile, projects it, syncs with other devices, and launches a non-blocking all-reduction via NVLink. While communication for query tile i is in flight, tile $i + 1$ is already being computed. The reason is possible is because the NVLink hardware is different from the CUDA cores and designed to operate in parallel.

4.2 FlashAttention Kernel

The attention kernel is organized around query tiles of B_M rows and key/value tiles of B_N rows. Each CUDA block handles one local head and one query tile at a time. A warp owns one query row, so a block uses $32B_M$ threads and requires $32B_M \leq 1024$. A single thread occupies a "lane" within the warp, which iterates OVER the key/value tiles in a single call. Threads load the current K_j and V_j tiles into shared memory, compute dot products, reduce partial scores with warp shuffles, update the online softmax state, and accumulate the output contribution from V_j . A structured description of the algorithm is provided.

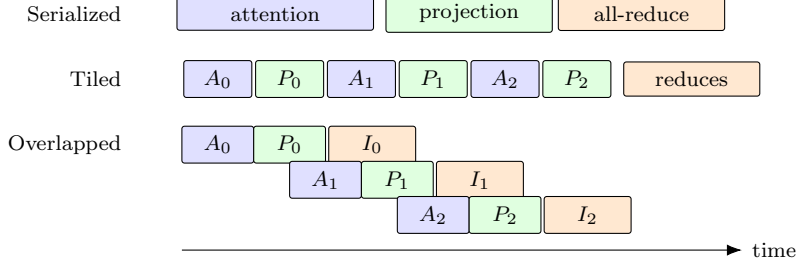


Figure 1: Execution modes. A_i is attention tile computation, P_i is output projection for tile i , and I_i is a nonblocking all-reduce for the projected tile.

The shared memory footprint per block is

$$2B_N d_h \cdot \text{sizeof}(\text{float}), \quad (10)$$

because each block stages one K_j tile and one V_j tile. The implementation assumes d_h and B_N are multiples of 32, since lanes divide the head dimension and score columns evenly. It also bounds per-thread arrays with maximum values for B_N and d_h , so final experiments use tile sizes that satisfy both shared memory limits and these shape constraints. The warp-per-row approach means that per-row outputs such as the row-wise softmax update and row reductions can use warp shuffles instead of shared memory (achieving a throughput approaching one shuffle per cycle).

The kernel writes an output tile in a layout compatible with the subsequent projection:

$$O_{\text{tile}}[B_M][H_{\text{local}}][d_h]. \quad (11)$$

This layout lets cuBLAS consume the tile for output projection without materializing the full sequence output first. The tradeoff is that smaller B_M produces earlier handoff points but less work per kernel launch.

4.3 Projection and Communication Pipeline

After an attention tile is finalized, the output projection computes

$$P_i^{(r)} = O_i^{(r)} W_O^{(r)}, \quad P_i^{(r)} \in \mathbb{R}^{B_M \times d}. \quad (12)$$

The projected tile is then reduced across ranks:

$$O_i = \sum_{r=0}^{R-1} P_i^{(r)}. \quad (13)$$

The implementation uses CUDA streams for attention computation, projection, and device-to-device result copies. Instead of using callbacks or other event-driven programming, we completely eliminate this overhead by allocating a separate CPU process for each device using MPI, taking advantage of the massive core count of Perlmutter node CPUs. Each process loops over each query tile i in order and launches a block of $B_M \cdot 32$ threads for each of its heads. After synchronizing, it performs the output projection with its shard $W_O^{(r)}$ using `cublasSgemm`, and then launches an asynchronous all-reduce over NVLink using NVIDIA Collective Communications Library (NCCL).

The `ncclAllReduce` returns immediately and the process immediately proceeds to the next query tile $i + 1$, only calling `cudaStreamSynchronize` after this next tile has been computed. In order to facilitate this overlap, each device keeps a buffer over the tiles.

Algorithm 1 flash_attn_kernel

Require: $Q, K, V \in \mathbb{R}^{H_{\text{local}} \times S \times d_h}$, `tile_idx`, `params`

Ensure: $O_{\text{tile}}[\text{warp}, h, :]$

```
1: lane = threadIdx.x mod 32,  c = dh/32;  each lane owns column slice [lane · c, (lane+1) · c)
2: q ← Q[h, tile_idx · BM + warp, lane's cols]
3: o ← 0,  m ← −∞,  ℓ ← 0

4: for j = 0, BN, ..., S − BN do
5:   Load K[h, j:j+BN], V[h, j:j+BN] into Ksmem, Vsmem;  __syncthreads()
6:   for i = 0 to BN − 1 do
7:     si ← scale · ⟨q, Ksmem[i]⟩ via partial sum then __shfl_xor_sync warp reduce
8:     Lane i mod 32 stores si into scores[i/32]
9:   end for
10:  mtile ← maxi scores[i] reduced across lanes via __shfl_xor_sync
11:  mnew ← max(m, mtile),  α ← em−mnew
12:  o ← α o,  ℓ ← α ℓ,  m ← mnew
13:  scores[i] ← escores[i]−m;  ℓ += ∑i scores[i] reduced via __shfl_xor_sync
14:  for i = 0 to BN − 1 do
15:    Pi ← __shfl_sync(scores[i/32], i mod 32);  o += Pi · Vsmem[i, lane's cols]
16:  end for
17:  __syncthreads()
18: end for
19: Otile[warp, h, lane's cols] ← o/ℓ
```

4.4 Correctness Validation

Correctness is evaluated by comparing against a straightforward single-GPU implementation on small inputs. The reference computes standard scaled dot-product attention and applies the output projection using cuBLAS kernels and offline softmax, and sums the rank-local projected outputs. We report maximum absolute and relative error. Exact bitwise equality is not expected because floating-point arithmetic is non-associative.

5 Experimental Methodology

Experiments are run on the NERSC Perlmutter compute cluster using up to 4x NVIDIA A100 GPUs connected via NVLink within a single node, and a 64-core AMD EPYC 7763 CPU. We evaluate sequence lengths $S \in \{2^{10}, 2^{12}, 2^{14}, 2^{16}\}$ with 16 heads for all experiments, along with a per-head dimension fixed at 64. All of these parameters were chosen based on typical transformer applications in modern use cases.

To analyze the influence of ranks and tiling on performance, we assessed rank counts of $R \in \{1, 2, 4\}$ and query tile sizes of $B_M \in \{8, 16, 32\}$, with B_N fixed at 64. The max query tile size is limited because we launch $B_M \cdot 32$ threads per block, and a single CUDA block can contain at most 1024 threads. For B_N , this value is chosen to maximize use of shared memory. As stated before, the shared memory footprint is $2B_N d_h \cdot \text{sizeof(float)}$. A single streaming multiprocessor has 48 KB of shared memory, so given the other parameters, we maximize use with $B_N = 64$.

The primary metric is end-to-end wall time. We feel this is the fairest metric because the

Table 1: Correctness validation of `flashattn_kernel` against baseline.

S	d	H	R	Max abs. err.
128	64	4	4	2.45×10^{-4}
256	128	8	4	2.62×10^{-4}
512	128	8	4	1.56×10^{-4}
512	256	8	4	2.11×10^{-10}
1024	256	8	4	1.97×10^{-10}

pipelined approach adds a reasonable amount of complexity that is not easy to directly measure. So just measuring whether the tradeoff is worth it can be achieved via end-to-end wall time. We use MPI barriers before initializing QKV and other state and after computing the output to make everything as fair as possible. For each mode, we run 3 warmup iterations followed by 5 trial iterations, and collect the min, max, median, mean, and standard deviation of the data.

We perform three groups of "variation" among the experiments:

1. **Mode comparison:** serialized, tiled non-overlapped, and tiled overlapped execution at fixed problem size.
2. **Strong scaling:** vary the number of tensor-parallel ranks while holding the global model configuration fixed.
3. **Tile-size sensitivity:** vary B_M and B_N to measure the tradeoff between earlier communication handoff and added scheduling overhead.

6 Results and Analysis

We evaluate the three execution modes across tile sizes, sequence lengths, and rank counts. All timings cover attention, output projection, and the all-reduce; QKV projection is outside the timed region.

6.1 Tile-Size Sensitivity

Table 2 reports wall time for query tile sizes $B_M \in \{8, 16, 32\}$ across all three modes at $R = 4$ ranks. The key/value tile size is fixed at $B_N = 64$, which fits comfortably within the default 48 KB per-block shared-memory budget for $d_h = 64$ while satisfying the kernel’s multiple-of-32 and fixed-array constraints.

Observation. Larger tiles are consistently faster. At $S = 1024$ the Megatron-style baseline drops from 61.6 ms ($B_M = 8$) to 24.0 ms ($B_M = 32$), roughly a $2.6\times$ speedup. The pattern holds across modes and sequence lengths: at $S = 4096$ the overlapped mode goes from 990.7 ms to 383.2 ms ($2.6\times$), and at $S = 65536$ from 341.4 s to 101.8 s ($3.4\times$). $B_M = 32$ is the best or near-best setting in every case.

Why larger tiles win. Every query tile carries fixed overhead: a kernel launch, a cuBLAS projection call, CUDA event synchronization, and a collective operation. Halving B_M doubles the tile count and therefore doubles that overhead, while the per-tile messages to the all-reduce get

Table 2: Wall time (mean $\pm$ std, ms) across modes and query tile sizes B_M , with $B_N = 64$, $R = 4$. The Megatron-style baseline uses the same tiled attention kernel, but computes all local attention before projection and one blocking all-reduce.

S	B_M	Megatron-style	Tiled-Sync	Overlap
1024	8	61.590 ± 0.018	66.876 ± 0.040	64.740 ± 0.079
	16	33.728 ± 0.042	36.629 ± 0.046	35.563 ± 0.018
	32	24.022 ± 0.038	25.358 ± 0.020	24.917 ± 0.139
4096	8	977.896 ± 0.035	999.440 ± 0.228	990.730 ± 0.120
	16	533.735 ± 0.080	545.953 ± 0.211	542.589 ± 0.047
	32	381.053 ± 2.062	385.109 ± 0.037	383.226 ± 0.072
16384	8	17824.756 ± 0.154	18122.368 ± 0.488	18391.905 ± 0.139
	16	9103.091 ± 0.033	9230.768 ± 0.166	9220.214 ± 0.181
	32	6277.922 ± 1.990	6260.489 ± 0.154	6256.563 ± 0.182
65536	8	324891.0 ± 0.6	328651.0 ± 2.0	341428.0 ± 0.6
	16	155225.9 ± 0.1	156071.0 ± 0.7	156679.0 ± 0.7
	32	101860.0 ± 4.0	101775.0 ± 0.6	101767.0 ± 0.5

proportionally smaller. In practice, amortizing these fixed costs matters more than creating extra overlap opportunities.

Overlap vs. tiled-sync. Comparing the last two columns of Table 2 lets us separate the cost of tiling from the benefit of async communication. At $B_M = 32$ and moderate S (≤ 4096), overlap is about 0.4–1.9 ms faster than tiled-sync, so some collective latency is genuinely hidden. But at large S with small tiles (e.g. $S = 65536$, $B_M = 8$) overlap is actually *slower* by 12.8 s (3.9%). The likely cause is that orchestration overhead—kernel launches, NCCL scheduling, stream contention, buffer management—adds up faster than the communication it hides, though the exact breakdown is hard to isolate from wall time alone.

6.2 Strong Scaling

Since $B_M = 32$ is clearly the best tile size, we fix it for all scaling experiments and vary the rank count $R \in \{1, 2, 4\}$. Figure 2 shows the results.

Observation. All three modes scale nearly linearly. At $S = 4096$ the Megatron-style baseline goes from 1566.4 ms ($R = 1$) to 384.1 ms ($R = 4$), a $4.08\times$ speedup on 4 ranks. Tiled-sync and overlap track closely, reaching 388.8 ms and 386.8 ms at $R = 4$ —within 1.2% and 0.7% of the Megatron-style time.

Modes converge at large S . As sequence length grows, the three modes become nearly indistinguishable. At $S = 65536$ and $R = 4$ they all land within 93 ms of each other (101,860, 101,775, 101,767 ms), a spread under 0.1%. This makes sense: within the timed region, attention compute scales quadratically in S (more query–key pairs) while the all-reduce only communicates the projected output, which scales linearly. So the communication fraction shrinks and there is less and less collective latency to hide.

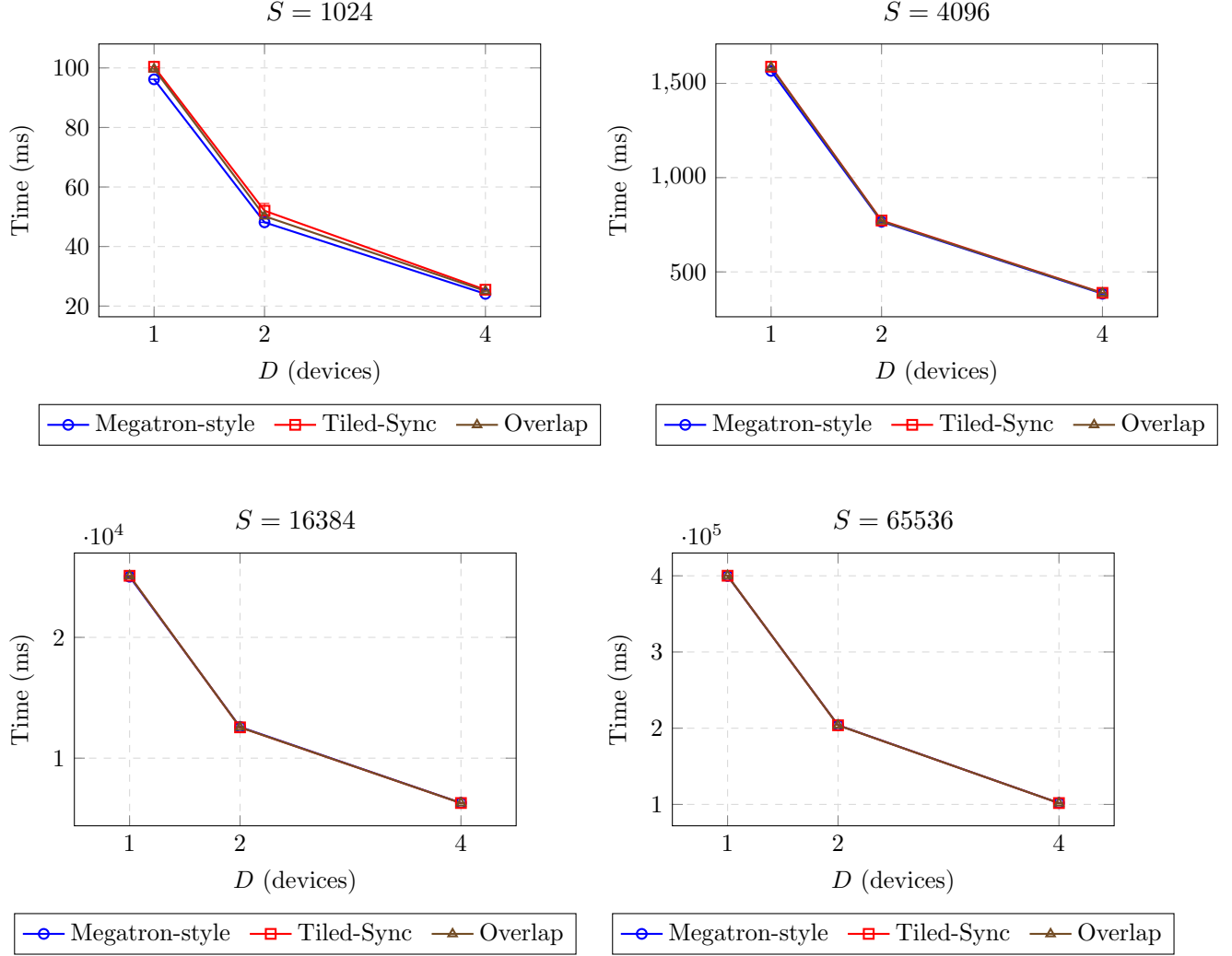


Figure 2: Strong scaling (mean $\pm$ std) across modes with $B_M = 32$, $B_N = 64$.

Small- S regime. The mode differences are most visible at short sequences. At $S = 1024$ and $R = 4$, the Megatron-style baseline finishes in 24.1 ms while tiled-sync takes 25.5 ms (5.8% slower) and overlap takes 25.1 ms (4.0% slower). Per-tile overhead is a real fraction of total runtime here, but overlap still recovers about a third of the tiling penalty.

6.3 Revisiting the Hypothesis

Our idealized model predicted $t_{\text{overlap}} \approx \max(t_{\text{compute}}, t_{\text{allreduce}})$ —that overlap should eliminate the additive communication cost. The data tells a more nuanced story.

The central issue is tile granularity. Smaller B_M creates more chances to overlap, but each tile carries less compute and a smaller message, so per-tile overhead adds up. Larger B_M amortizes that overhead and gives NVLink bigger messages to work with, but leaves fewer overlap windows. In our experiments, amortization consistently won: $B_M = 32$ was the best legal tile size, and the overlap benefit over tiled-sync was modest (0.4–1.9 ms at moderate S). At large S , compute dominates regardless and all modes converge.

That said, the implementation does show that the pipeline is mechanically sound: tiles are

finalized, projected, and reduced asynchronously without correctness loss. The bottleneck is not the overlap idea itself but the narrow window in which it can pay off—small enough S that communication matters, but large enough tiles that overhead stays low. Our one-warp-per-row mapping caps B_M at 32 and keeps that window tight. Widening it, for example by having a warp handle multiple query rows, is the most promising direction for making overlap worthwhile.

7 Challenges and Obstacles

Dependency management. The main implementation challenge is that the algorithm combines CUDA kernels, cuBLAS projections, CUDA stream/event dependencies, and collective communication. Correctness requires that projection never reads an unfinished attention tile and communication never reads an unfinished projected tile. Performance requires preserving those dependencies without unnecessary device-wide synchronizations.

Communication backend. An early version of the pipeline used CPU-side OpenMPI collectives, which was useful for validating correctness but too slow for the overlap experiment. The communication path itself became the bottleneck, so measured timings reflected host/MPI overhead more than GPU overlap behavior. Moving to NCCL was necessary because NCCL operates directly on device buffers and can use NVLink more effectively.

Kernel shape constraints. The FlashAttention kernel has strict shape requirements. The prototype maps one query row to a warp, so d_h and B_N interact with warp-level reductions and fixed-size local arrays. Shared memory usage scales with $2B_N d_h$, since each block stages one key tile and one value tile. These constraints limit which tile shapes are legal and which give high occupancy.

Collective progress and overlap measurement. Asynchronous collective launches do not by themselves guarantee ideal overlap. Progress behavior depends on the backend, CUDA-aware transport support, and how the host thread interacts with outstanding requests. As a result, measuring true communication overlap is more subtle than measuring total wall time.

System and environment issues. On Perlmutter, CUDA-aware Cray MPICH required explicit GPU-support environment settings, and CUDA IPC behavior could affect whether device-buffer collectives were stable. Timing was sensitive to first-call initialization costs in CUDA, cuBLAS, and NCCL, so repeated runs and warmup iterations were needed to avoid overinterpreting a single measurement. Result collection was also constrained by the need to coordinate kernel fixes, batch allocation availability, and consistent problem configurations across modes.

External comparison. Direct comparison against external attention implementations is difficult because performance would reflect both our communication schedule and lower-level kernel differences. Our FlashAttention-style tiling can improve locality by staging key/value tiles in shared memory, which may produce speedups independent of communication overlap. Production kernels may also use different work partitioning, memory layouts, fusion strategies, or tensor-core paths. To isolate the effect of overlap, we compare modes that all use the same attention kernel.

Warp-mapping limitation. A key limitation of our prototype is the one-warp-per-query-row mapping. This design makes the row-wise softmax update simple—each row’s reductions use warp shuffles—but it ties the query tile size directly to the CUDA thread-block limit: a block for B_M rows needs $32B_M$ threads, and CUDA blocks max out at 1024, so B_M is capped at 32. We could not test larger tiles even though they might better amortize projection and collective overhead. A promising fix is to decouple tile size from this mapping, for example by having a warp process multiple rows or assigning each row to a smaller group of lanes.

References

- [1] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053, 2019.
- [2] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *SC*, 2021.
- [3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.
- [4] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691, 2023.
- [5] Bin Xiao and Lei Su. ISO: Overlap of Computation and Communication within Sequence For LLM Inference. arXiv:2409.11155, 2024.